

Universidad Gerardo Barrios
Facultad de Ciencia y Tecnología

Informe final de proyecto

Trabajo de graduación — Modalidad preespecialización en la
especialidad de:

Desarrollo de Aplicaciones con Inteligencia Artificial

“VeriNewsIA & Phishing”

Extensión de navegador con backend de IA para la detección de
phishing en tiempo real

Presentado por:

Melvin Santiago Rodríguez Mendoza

Katia Guadalupe Orellana Romero

Eskin Onan Montiel Hernández

San Miguel, 22 de agosto de 2026

Índice

Resumen del proyecto:	3
Marco conceptual:	4
1. Phishing y detección automatizada de URLs	4
2. Aprendizaje automático aplicado a la clasificación de URLs	5
2.1. Random Forest	5
2.2. Transformers y RoBERTa	5
2.3. Fusión de modelos (ensembling)	6
3. Motores de reglas explicables (scoring aditivo)	6
4. Inteligencia de amenazas (Threat Intelligence)	6
5. Arquitectura de API REST con FastAPI	7
6. Seguridad de aplicaciones web expuestas a entrada no confiable	7
7. Extensiones de navegador (Manifest V3)	7
8. Prácticas de ingeniería: contenedores, CI/CD y observabilidad	8
Análisis del problema:	8
Propuesta de solución:	9
Metodología de desarrollo del proyecto:	11
1. API inteligente y contratos	11
2. Pruebas y CI/CD	12
3. Despliegue e infraestructura	13
Resultados alcanzados	13
1. Observabilidad, rendimiento y escalabilidad	13
2. Seguridad, release y rollback	15
Conclusiones generales:	16
Referencias:	17

Resumen del proyecto:

VeriNewsIA & Phishing es una extensión para navegadores basados en Chromium (Google Chrome, Microsoft Edge), construida sobre Manifest V3, que permite a cualquier usuario verificar en tiempo real si una URL representa un riesgo de phishing antes de introducir credenciales o datos sensibles en ella. El problema que atiende es concreto: el phishing sigue siendo uno de los vectores de ataque más usados para el robo de credenciales, y la mayoría de los usuarios no cuenta con una herramienta rápida, explicable y gratuita para evaluar un enlace sospechoso antes de interactuar con él.

El usuario objetivo es la persona que navega activamente por internet (correo, redes sociales, resultados de búsqueda) y necesita una segunda opinión inmediata sobre un enlace antes de decidir si confiar en él sin conocimientos técnicos previos, y sin instalar software adicional más allá de la extensión.

La función de la inteligencia artificial dentro del producto es doble. Por un lado, dos modelos de Machine Learning entrenados específicamente para este dominio, un Random Forest sobre 34 características extraídas de la URL y el HTML, y un modelo RoBERTa afinado (fine-tuned) sobre el texto crudo de la URL, se combinan mediante una fusión ponderada (40 % / 60 %) para producir una probabilidad de phishing. Por otro lado, un tercer modelo RoBERTa, especializado en clasificación de contenido (noticias reales vs. falsas), está disponible como servicio independiente para evaluar el texto de una página. Estas señales de IA se combinan con reglas heurísticas explícitas (antigüedad del dominio, presencia de IP en vez de dominio, símbolos de engaño, TLD sospechosos) y con tres fuentes externas de inteligencia de amenazas (VirusTotal, Google Safe Browsing y Google Fact Check Tools) dentro de un motor de riesgo aditivo y auditable, que entrega un puntaje de 0 a 100, un veredicto LOW / MEDIUM / HIGH y una lista de razones en lenguaje natural que explican la decisión.

La propuesta de valor frente a alternativas existentes es la combinación de explicabilidad (cada veredicto se acompaña de sus razones y de una confianza calibrada), degradación controlada (si una API externa o un modelo falla, el resto

de las señales sigue funcionando en vez de tumbar el análisis completo) y un pipeline paralelo que entrega una respuesta completa en 3 a 8 segundos. El resultado del proyecto durante las seis sesiones del módulo es un backend en FastAPI con 11 endpoints documentados, una batería de 535 pruebas automatizadas, un pipeline de integración continua en GitHub Actions, una imagen Docker desplegable y una extensión de navegador completa (popup, panel lateral y página de configuración), junto con un ciclo de retroalimentación del usuario que sienta la base para el reentrenamiento futuro de los modelos con datos reales.

Marco conceptual:

1. Phishing y detección automatizada de URLs

El phishing es una técnica de ingeniería social en la que un atacante suplanta la identidad de una entidad confiable (un banco, una red social, un servicio de correo) para engañar a la víctima y obtener credenciales, datos financieros o acceso a sistemas. A diferencia del malware tradicional, el phishing no depende de explotar una vulnerabilidad de software: depende de que el usuario tome una decisión de confianza equivocada frente a una URL, un correo o una página que imita a la legítima. Por eso la mayoría de las defensas automatizadas — incluida la de este proyecto se concentran en analizar la URL y la página de destino antes de que el usuario interactúe con ella, en vez de intentar detectar la intención del atacante directamente.

Existen dos grandes familias de enfoques para esta detección. La primera es la basada en listas negras (blocklists): un dominio se marca como malicioso una vez que ha sido reportado, y los servicios de terceros (VirusTotal, Google Safe Browsing) distribuyen esa lista. Su ventaja es la precisión casi perfecta una vez que un sitio está catalogado; su debilidad es la ventana de tiempo antes de esa catalogación, en la que un dominio de phishing recién creado pasa inadvertido. La segunda familia es la basada en heurísticas y aprendizaje automático: en vez de esperar un reporte, se extraen características de la URL, del dominio y del contenido de la página, y un modelo o un conjunto de reglas estima la probabilidad de que se trate de un sitio fraudulento. Este proyecto combina ambos enfoques

deliberadamente: los modelos de ML y las heurísticas cubren la ventana temprana en la que un dominio aún no ha sido reportado, y las APIs de threat intelligence aportan certeza absoluta cuando el sitio ya es conocido.

2. Aprendizaje automático aplicado a la clasificación de URLs

2.1. Random Forest

Un Random Forest es un modelo de ensamble compuesto por muchos árboles de decisión entrenados sobre subconjuntos aleatorios de los datos y de las variables (bagging), cuya predicción final es el promedio (o el voto mayoritario) de todos los árboles individuales. Se eligió para este proyecto porque trabaja de forma natural con datos tabulares heterogéneos — en este caso, 34 características numéricas y booleanas extraídas de la URL y del HTML de la página (longitud de la URL, cantidad de subdominios, presencia de un formulario de contraseña, cantidad de scripts, etc.) —, es comparativamente económico de entrenar e inferir frente a un modelo de lenguaje, y permite inspeccionar la importancia relativa de cada característica, lo que ayuda a auditar por qué el modelo decide lo que decide. El modelo entrenado (“random_forest_v2.pk”, con su orden de columnas fijado en “feature_columns_v2.pkl”) usa la convención de etiquetas 0 = phishing / 1 = legítimo; la probabilidad de phishing se toma de la clase 0 y la de sitio legítimo de la clase 1.

2.2. Transformers y RoBERTa

RoBERTa es una variante optimizada de BERT, un modelo de lenguaje basado en la arquitectura Transformer, que aprende representaciones contextuales de texto mediante mecanismos de auto-atención (self-attention): a diferencia de un modelo tabular, no necesita que alguien defina a mano qué características observar, sino que aprende directamente patrones sobre la secuencia de caracteres o palabras. El proyecto usa dos instancias distintas de este modelo, cada una afinada (fine-tuning) sobre un corpus distinto: una versión ligera (“distilroberta-base”) fine-tuneada directamente sobre cadenas de URL para capturar patrones léxicos propios del phishing (subdominios engañosos, combinaciones de palabras como “verify”, “secure”, “login”, sustituciones de caracteres), y una segunda instancia fine-

tuneada sobre un corpus de noticias reales y falsas (`GonzaloA/fake\_news`) para la clasificación de contenido, expuesta como servicio independiente.

2.3. Fusión de modelos (ensembling)

El “FusionEngine” combina la probabilidad de phishing del Random Forest y la del RoBERTa de URL con una ponderación fija (40 % / 60 % a favor de RoBERTa), en vez de promediar sin más. La justificación es que un modelo de lenguaje entrenado directamente sobre el texto de la URL captura mejor los patrones léxicos del phishing moderno que un conjunto fijo de características tabulares, aunque ambos aportan señal independiente. El componente degrada con elegancia: si uno de los dos modelos falla en tiempo de ejecución, el otro pasa a operar con el 100 % del peso en vez de propagar el error.

3. Motores de reglas explicables (scoring aditivo)

En vez de delegar la decisión final íntegramente en un modelo de caja negra, el proyecto usa un motor de riesgo (RiskEngine) basado en un modelo aditivo: parte de un puntaje neutro de 50 sobre 100 y aplica penalizaciones o bonificaciones explícitas por cada señal detectada (HTTPS, edad del dominio, presencia de un formulario de contraseña, resultado de cada API externa, resultado de la fusión de modelos, etc.), hasta acotar el resultado al rango [0, 100]. Este diseño prioriza la explicabilidad (Explainable AI): cada veredicto se puede descomponer en la lista exacta de razones que lo produjeron, algo que un único modelo de caja negra no ofrece por sí solo. Sobre ese puntaje se añade además una capa de calibración (`confidence_calibration.py`) que traduce el score a una probabilidad mediante una función sigmoide y reporta un nivel de confianza explícito, que baja cuando los modelos discrepan entre sí o cuando pocas señales contribuyeron al resultado.

4. Inteligencia de amenazas (Threat Intelligence)

Las APIs de threat intelligence integradas, VirusTotal API v3, Google Safe Browsing API v4 y Google Fact Check Tools API, son bases de datos colaborativas mantenidas por terceros que agregan reportes de múltiples motores antivirus y de múltiples fuentes de verificación de datos. Su función en el pipeline no es predecir, sino confirmar: cuando un dominio ya fue reportado como malicioso, estas fuentes

lo detectan con una precisión muy superior a la de cualquier modelo entrenado localmente, y por eso sus señales tienen el mayor peso negativo dentro del “RiskEngine” (hasta 50 puntos ante una detección de Safe Browsing).

5. Arquitectura de API REST con FastAPI

El backend expone su funcionalidad como una API REST construida con FastAPI, un framework de Python asíncrono elegido por tres razones: valida automáticamente el contrato de entrada y salida mediante modelos Pydantic (lo que impide, por ejemplo, que una URL sin prefijo “http”/“https” llegue a procesarse), genera de forma automática documentación interactiva (OpenAPI / Swagger) a partir del propio código, y soporta de forma nativa operaciones asíncronas y paralelas esenciales para un pipeline que dispara seis llamadas de E/S en paralelo (WHOIS, HTML, tres APIs externas y un modelo de ML) por cada análisis.

6. Seguridad de aplicaciones web expuestas a entrada no confiable

Como el backend recibe URLs arbitrarias controladas por el usuario y las descarga (HtmlFetcher), el diseño tuvo que incorporar defensas específicas contra Server-Side Request Forgery (SSRF): cada hostname se resuelve y se rechaza si alguna de sus IPs es privada, loopback, link-local o pertenece a un endpoint de metadatos de nube (169.254.169.254), las redirecciones se siguen manualmente re-validando el host en cada salto, y el cuerpo de la respuesta se limita a 2 MB. A esto se suman un límite de tasa por IP (RateLimitMiddleware, 30 solicitudes por minuto), un cortacircuitos (circuit breaker) que protege la cuota de VirusTotal, autenticación opcional por cabecera “X-API-Key” comparada con “hmac.compare_digest” para evitar ataques de temporización, y una política de CORS restringida por expresión regular a orígenes “chrome-extension://” y “localhost”.

7. Extensiones de navegador (Manifest V3)

Manifest V3 es el modelo de extensiones vigente en navegadores basados en Chromium. Reemplaza los scripts de fondo persistentes por service workers efímeros, prohíbe la ejecución de código remoto (todo el JavaScript debe

empaquetarse dentro de la extensión) y exige declarar explícitamente cada permiso solicitado. El proyecto aprovecha ese modelo restrictivo a su favor: no solicita los permisos “tabs” ni “activeTab”, por lo que nunca analiza una pestaña automáticamente — todo análisis es iniciado explícitamente por el usuario — y el ``background.js`` se limita a un chequeo periódico de salud del backend.

8. Prácticas de ingeniería: contenedores, CI/CD y observabilidad

El backend se distribuye como imagen Docker (python:3.12-slim), que descarga los pesos de los tres modelos desde Hugging Face Hub durante la construcción de la imagen (build time), de modo que el contenedor final no depende de una descarga en tiempo de ejecución. La integración continua (GitHub Actions) ejecuta en cada push o pull request a “main” el linter (“ruff”) y la batería completa de pruebas automatizadas contra un backend con los modelos simulados (mockeados), de forma que la suite no necesita los archivos de peso reales para validar la lógica de negocio. Un segundo flujo de trabajo (smoke-test.yml) verifica, después de cada despliegue, que el endpoint `/predict` responde de verdad con un puntaje — porque `/health` y `/metadata` pueden devolver 200 incluso cuando la inferencia real está fallando por falta de memoria. La observabilidad local se apoya en Prometheus y Grafana, con métricas de conteo de solicitudes, histogramas de latencia y duración de arranque expuestas en `/metrics`.

Análisis del problema:

El diagnóstico inicial del proyecto (sesión 1) identificó que un usuario común no dispone de una forma rápida de verificar la legitimidad de una URL antes de interactuar con ella: las defensas existentes viven del lado del proveedor de correo o del propio navegador, no son transparentes en sus razones, y no ofrecen al usuario una segunda opinión bajo demanda. Los riesgos iniciales identificados fueron tres: (1) depender de una sola fuente de verdad (por ejemplo, solo un modelo de ML) generaría falsos positivos y falsos negativos difíciles de auditar; (2) un backend que analiza URLs arbitrarias enviadas por un cliente no confiable (la extensión, distribuida por cualquiera) está expuesto a abuso (SSRF, agotamiento de cuota de APIs externas, scraping); y (3) los modelos de Machine Learning tienen

un costo de memoria significativo, que en un entorno de despliegue gratuito y acotado (512 MB) podía volverse el cuello de botella del propio backend.

La deuda técnica del MVP inicial (2026-06-03) era considerable: el Random Forest inicial usaba solo 14 características, no existía fusión entre modelos, no había integración con ninguna API de threat intelligence más allá de VirusTotal y Safe Browsing, la extensión no tenía panel lateral ni página de configuración, y no existía batería de pruebas automatizadas. La evolución documentada en el historial de cambios del proyecto muestra un camino incremental: incorporación del clasificador RoBERTa de URLs y del `FusionEngine` (junio), interfaz completa de la extensión con popup, sidebar y opciones (junio), corrección de un error crítico en el Random Forest que invertía en silencio las probabilidades de phishing/legítimo (afectando el 40 % del resultado de la fusión), incorporación del guardián SSRF, monitorización con Prometheus/Grafana y trazado con OpenTelemetry (agosto), calibración del motor de riesgo con probabilidad, intervalos de confianza y concordancia entre modelos (agosto), y finalmente la resolución de un fallo de memoria en producción causado, de forma contraintuitiva, por la cuantización int8 que se había introducido para acelerar la inferencia (agosto). Esa última corrección documentada con detalle en el registro de cambios del proyecto es un ejemplo directo de deuda técnica reconocida y resuelta con evidencia medida, no supuesta.

Propuesta de solución:

La solución adoptada separa el sistema en dos componentes independientes que se comunican por HTTP: una extensión de navegador (cliente, sin lógica de decisión propia) y un backend de análisis (donde vive toda la inteligencia). Esta separación se eligió sobre la alternativa de un modelo embebido en el propio navegador porque los modelos de ML usados (RoBERTa, en particular) son demasiado pesados para ejecutarse razonablemente dentro de una extensión, y porque mantener los modelos del lado del servidor permite actualizarlos sin republicar la extensión en cada tienda.

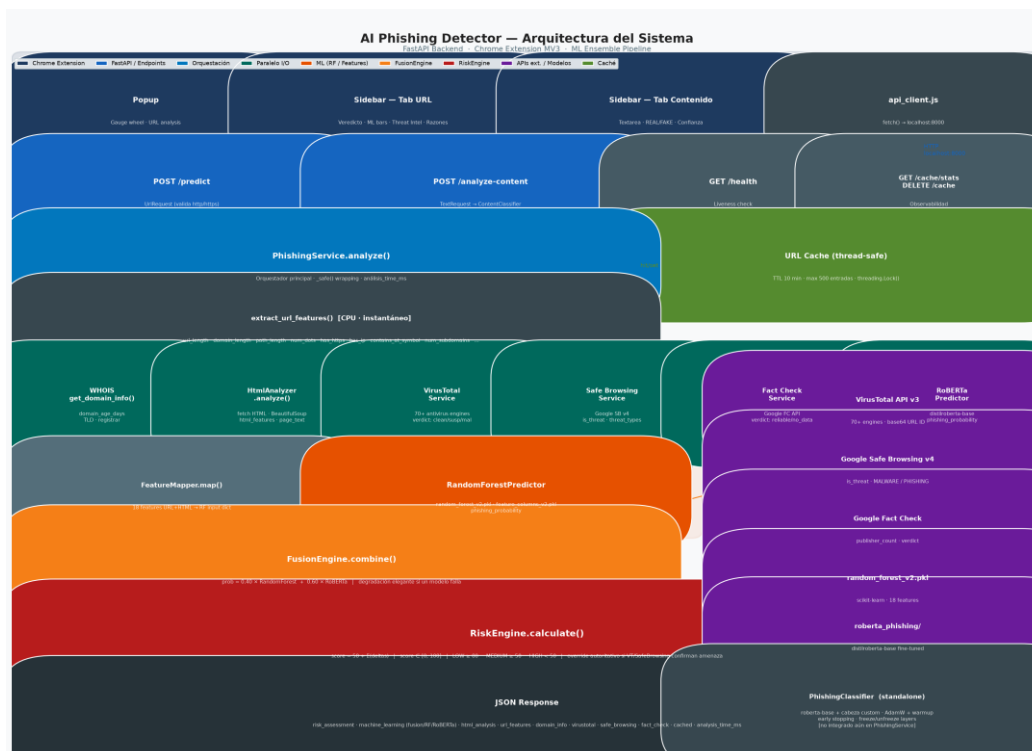


Figura 1. Arquitectura del pipeline de análisis (docs/architecture_diagram.png).

El pipeline de análisis (PhishingService.analyze) se organiza en dos oleadas paralelas seguidas de una etapa secuencial. La extracción de características de la URL (12 features, solo CPU) corre primero. Después, un “ThreadPoolExecutor” de 6 hilos ejecuta en paralelo: consulta WHOIS del dominio, descarga y análisis del HTML, consulta a VirusTotal, consulta a Google Safe Browsing, consulta a Google Fact Check y la inferencia del clasificador RoBERTa de URL. Una segunda etapa ejecuta el Random Forest, que depende del resultado del HTML de la etapa anterior. Finalmente, de forma secuencial, el FusionEngine combina ambos modelos de ML y el ‘RiskEngine’ agrega todas las señales en un puntaje final con sus razones. El resultado se cachea en memoria durante 10 minutos para no repetir trabajo ni gastar cuota de API ante la misma URL.

Se eligió deliberadamente un motor de riesgo aditivo y explicable (RiskEngine) por encima de delegar la decisión final a un único modelo de caja negra, precisamente porque la evaluación exige poder explicar decisiones y reconocer riesgos: cada componente del puntaje final queda trazado a una señal concreta, auditable línea por línea en (docs/decision_tree.md).

Metodología de desarrollo del proyecto:

1. API inteligente y contratos

El backend expone 11 endpoints bajo un contrato tipado con modelos Pydantic tanto de entrada (UrlRequest, TextRequest) como de salida (un response_model distinto por endpoint, documentado en el esquema OpenAPI). Solo cuatro rutas son públicas; el resto exige la cabecera “X-API-Key” cuando el backend la tiene configurada.

Método	Ruta	Descripción	Auth
POST	/predict	Análisis completo de una URL (pipeline multi-señal)	Sí*
POST	/analyze-content	Clasificación de texto libre (REAL/FAKE)	Sí*
POST	/feedback	Registrar una corrección del usuario sobre un veredicto	Sí*
GET	/feedback/stats	Correcciones acumuladas (falsos positivos/negativos)	Sí*
GET	/experiment/status	Configuración activa del experimento de scoring	Sí*
GET	/cache/stats	Estadísticas del cache en memoria	Sí*
DELETE	/cache	Vaciar el cache	Sí*
GET	/	Comprobación básica de disponibilidad	No
GET	/health	Liveness check	No
GET	/metadata	Versión, modelos presentes y configuración activa	No
GET	/metrics	Métricas Prometheus (fuera del esquema OpenAPI)	No

*La autenticación es un no-op cuando `API\_KEY` está vacío. En este despliegue está deliberadamente vacío y declarado como público (`ALLOW\_UNAUTHENTICATED=true`), porque la extensión se distribuye por una tienda y su paquete es descomprimible por cualquiera: una clave embebida en el cliente sería pública desde el primer día. La protección real del backend público

recae en el límite de tasa por IP, el cortacircuitos de cuota de VirusTotal y el guardián SSRF.

Cada solicitud inválida (por ejemplo, una URL sin prefijo `http`/`https`) es rechazada por la validación de Pydantic con `422 Unprocessable Entity` antes de tocar cualquier lógica de negocio. Un manejador de excepciones global captura cualquier error no controlado, lo registra con su traza completa y devuelve `500` sin exponer detalles internos cuando el entorno es de producción.

2. Pruebas y CI/CD

La suite automatizada cuenta actualmente con 535 pruebas (pytest), repartidas entre “backend/tests/” y “tests/”, y corre sin necesidad de los archivos de modelo reales (“.pkl” ni directorios de Hugging Face): un “conftest.py” compartido sustituye los cargadores de modelo antes de que se importe el backend, y limpia entre pruebas el estado global del limitador de tasa y del cortacircuitos de VirusTotal.

Área cubierta	Qué valida
Guardián SSRF	34 pruebas — validación de IPs privadas/reservadas, prevención de DNS-rebinding
Límite de tasa	7 pruebas — cabeceras de proxy, bucketing por IP, evicción del store
Cortacircuitos VirusTotal	9 pruebas — agotamiento de cuota, reinicio, degradación controlada
Seguridad general	8 pruebas — filtrado de errores, ausencia de fuga de internals
Pipeline completo	200+ pruebas — features de URL/HTML, fusión, calibración, motor de riesgo

El flujo de integración continua (.github/workflows/ci.yml) instala las dependencias con la rueda de PyTorch para CPU (“requirements-dev.txt”, distinto del “requirements.txt” de producción que fija CUDA), ejecuta “ruff check .” y luego “pytest -v” en cada push o pull request contra “main”. Un segundo flujo (smoke-test.yml) se dispara después de cada despliegue y llama al “/predict” real del servicio publicado no a “/health” porque “/metadata” solo comprueba que los archivos de modelo existan en disco, sin decir nada sobre si la inferencia efectivamente cabe en memoria.

3. Despliegue e infraestructura

El backend se despliega como contenedor Docker en Render, a partir de ``backend/Dockerfile`` (imagen base ``python:3.12-slim``), con auto-deploy habilitado en cada push a ``main``. Durante la construcción de la imagen se descargan desde Hugging Face Hub (``mel3601/phishing-ia-models``) los tres modelos entrenados (~821 MB en total: Random Forest 25 MB, RoBERTa URL 317 MB, RoBERTa Content 479 MB), que quedan horneados en ``/models``. La carga de cada modelo en memoria es perezosa (``@lru_cache``), de modo que el primer ``/predict`` tras un reinicio paga el costo de carga (~10-20 s) y las solicitudes siguientes lo reutilizan.

Las variables de entorno relevantes (``VIRUSTOTAL_API_KEY``, ``SAFE_BROWSING_API_KEY``, ``FACT_CHECK_API_KEY``, ``API_KEY``, ``ALLOWED_ORIGINS``, ``ENVIRONMENT``, ``ALLOW_UNAUTHENTICATED``, ``MAX_CONCURRENT_ANALYSES``, ``ANALYSIS_QUEUE_TIMEOUT``, ``FORWARDED_ALLOW_IPS``) se configuran en el panel de Render y están documentadas, sin valores reales, en ``.env.example`` y en ``docs/DEPLOYMENT.md``. La reproducibilidad de la configuración no depende de pasos manuales no documentados: el Dockerfile y el flujo de despliegue son el mismo para cualquier colaborador que clone el repositorio.

Resultados alcanzados

1. Observabilidad, rendimiento y escalabilidad

El arranque en frío (cold start) del servicio, tras 15 minutos de inactividad en el plan gratuito de Render, toma entre 60 y 90 segundos: 5-10 s de arranque del contenedor, 10-20 s de carga perezosa de los modelos y ~5 s de arranque de FastAPI. Con los modelos ya cargados en memoria, una solicitud típica a ``/predict`` se descompone así:

Etapas	Duración aproximada
Extracción de características de URL	~100 ms
Descarga y análisis del HTML (si aplica)	+500-1500 ms
Inferencia de ML (en paralelo)	+200-500 ms

Etapas	Duración aproximada
Agregación del RiskEngine	~100 ms
Total típico	3-8 s (dominado por E/S de red, no por cómputo)

El cuello de botella identificado no fue la latencia sino la memoria: en el plan gratuito de Render (512 MB, un solo worker), dos análisis concurrentes hacían que el proceso superara el límite y el kernel lo terminara ('OOM-kill'), lo que además tumbaba '/health' junto con '/predict' (502 en producción). La causa raíz medida fue contraintuitiva: la cuantización int8 del modelo RoBERTa, introducida para acelerar la inferencia en CPU, en realidad materializaba todos los pesos del modelo en el proceso de conversión, generando un pico de +735 MB de RAM para ganar apenas 1,96 ms por URL en un pipeline de 3-8 s dominado por red. Al retirarla, el consumo de '/predict' bajó de 1213 MB a 791 MB. Como control adicional se añadió un semáforo de concurrencia ('MAX_CONCURRENT_ANALYSES', por defecto 1) que serializa los análisis en vez de dejar que la memoria pico se multiplique por solicitudes simultáneas. El plan de escalabilidad para tráfico mayor al actual contempla, en orden de costo: subir a un plan de Render con más memoria, entrenar un Random Forest con menos árboles (reduciendo su huella de 156 MB) una vez exista un conjunto de validación etiquetado que justifique el compromiso con la precisión, y si el tráfico lo amerita separar el servicio en más de una instancia detrás de un balanceador.

La observabilidad local se implementa con Prometheus ("/metrics", fuera del esquema OpenAPI) y un panel de Grafana provisionado por "docker-compose.yml", que exponen conteo de solicitudes por endpoint y código de estado (http_requests_total), histograma de latencia (http_request_duration_seconds) y duración de arranque (app_startup_duration_seconds). Cada solicitud queda además registrada por el "RequestLoggingMiddleware" con método, ruta, código de estado, latencia e IP de origen, lo que permite correlacionar un incidente puntual con su traza en los logs.

2. Seguridad, release y rollback

Riesgo	Control implementado
SSRF vía URL controlada por el usuario	Resolución y validación de IP por hostname, redirecciones re-validadas en cada salto, límite de 5 saltos y 2 MB de respuesta
Abuso / agotamiento de cuota de terceros	Límite de 30 solicitudes/60 s por IP; cortacircuitos dedicado para la cuota de VirusTotal
Comparación insegura de la API key	"hmac.compare_digest" en vez de "==", para evitar un canal lateral por temporización
Fuga de datos de navegación del usuario	Las URLs corregidas por el usuario se guardan hasheadas (SHA-256) en "feedback_store.py", nunca en claro
Fuga de detalles internos ante un error	El campo "detail" se omite en las respuestas de error cuando "ENVIRONMENT=production"
Arranque accidental sin autenticación	"ENVIRONMENT=production" sin "API_KEY" aborta el arranque, salvo declaración explícita "ALLOW_UNAUTHENTICATED=true"
Código remoto en la extensión (rechazado por las tiendas)	Manifest V3 lo prohíbe; se retiró un script cargado desde CDN que las tiendas habrían rechazado

Los casos adversariales se validan tanto en pruebas manuales documentadas (URL con IP en vez de dominio, URL con símbolo "@" para redirección engañosa, dominio con múltiples palabras clave de phishing, todos clasificados correctamente como HIGH) como en la suite automatizada del guardián SSRF. La verificación de humo post-despliegue (scripts/smoke_test.py) llama al "/predict" real, no al "/health", precisamente porque un backend con los modelos rotos puede seguir devolviendo 200 en los endpoints superficiales.

El release demostrado quedó etiquetado como ****v1.0-final**** (GitHub Release, https://github.com/melsysdev-web/phishing_ia/releases/tag/v1.0-final), asociado al commit `0d46f91` de `main` — el mismo estado de código descrito en todo este informe. Sus notas documentan qué incluye la versión y sus limitaciones conocidas, con trazabilidad directa al commit exacto.

El mecanismo de rollback disponible es el propio de Render sobre despliegues por Git: cada despliegue queda asociado a un commit, y volver a una versión anterior implica re-desplegar el commit previo o revertirlo en “main”. Los límites operativos conocidos del sistema en este momento son: un único worker por instancia (WEB_CONCURRENCY=1), como máximo un análisis concurrente por defecto (MAX_CONCURRENT_ANALYSES=1), 512 MB de memoria disponible, y un límite de 30 solicitudes por minuto por IP compartido entre /predict y “/analyze-content”.

Conclusiones generales:

El proyecto entrega un sistema de detección de phishing funcional, explicable y desplegado, que combina aprendizaje automático, reglas heurísticas auditables e inteligencia de amenazas de terceros dentro de un pipeline que degrada con control ante el fallo de cualquiera de sus señales. Más allá del producto en sí, el desarrollo dejó aprendizajes concretos de ingeniería: que la optimización de inferencia (cuantización) puede tener el efecto contrario al esperado bajo restricciones de memoria si no se mide directamente el consumo pico; que exponer un backend a URLs arbitrarias enviadas por un cliente no confiable exige defensas SSRF explícitas y no un simple “requests.get”; y que empaquetar una extensión de navegador para una tienda tiene reglas de validación (ubicación del manifiesto, ausencia de código remoto) que no aparecen al probar la extensión sin empaquetar.

Las limitaciones se declaran sin ocultarlas, tal como exige la evaluación. La más relevante es de datos: no existe todavía un conjunto de validación etiquetado independiente contra el cual medir la precisión real de los modelos, por lo que las mejoras de scoring implementadas se concentran en calibración (que la confianza reportada refleje mejor la certeza real) y en reducir una fuente conocida de falsos positivos (el dominio recién registrado), en vez de en una cifra de accuracy que no podría respaldarse con evidencia. Varias mejoras del plan original pesos aprendidos por señal, un ensemble apilado, fine-tuning adicional de RoBERTa sobre más datos quedaron deliberadamente sin implementar por la misma razón, y su desbloqueo depende de acumular correcciones reales de usuarios a través de “POST /feedback”, mecanismo que sí quedó construido y funcionando. Otras limitaciones conocidas son: el backend de producción es intencionalmente público

(sin autenticación real), porque una extensión distribuida en una tienda no puede resguardar un secreto; la capacidad del plan gratuito de despliegue es acotada (un solo worker, un análisis concurrente); y, como se señala en la sección anterior, el tag/release de Git y la URL pública definitiva a demostrar quedan pendientes de fijar antes del cierre.

Referencias:

- FastAPI — Documentación oficial. <https://fastapi.tiangolo.com/>
- scikit-learn — Random Forest Classifier. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>
- Hugging Face — Transformers y RoBERTa. https://huggingface.co/docs/transformers/model_doc/roberta
- Liu, Y. et al. — RoBERTa: A Robustly Optimized BERT Pretraining Approach (arXiv:1907.11692).
- Vaswani, A. et al. — Attention Is All You Need (arXiv:1706.03762).
- Google — Safe Browsing API (v4). <https://developers.google.com/safe-browsing>
- Google — Fact Check Tools API. <https://developers.google.com/fact-check/tools/api>
- VirusTotal — API v3 Documentation. <https://docs.virustotal.com/reference/overview>
- OWASP — Server-Side Request Forgery (SSRF) Prevention Cheat Sheet. <https://cheatsheetseries.owasp.org/>
- Chrome for Developers — Manifest V3. <https://developer.chrome.com/docs/extensions/mv3/intro/>
- Render — Docker Deploys Documentation. <https://render.com/docs/docker>
- Prometheus — Documentación oficial. <https://prometheus.io/docs/>

- Documentación interna del proyecto: docs/architecture.md, docs/decision_tree.md, docs/api.md, docs/DEPLOYMENT.md, docs/TESTING.md, SCORE_IMPROVEMENTS_STRATEGY.md.